



EXAMEN DEVOPS – RAPPORT DE PROJET

# Bibliothèque Numérique DIT

Conception, développement et automatisation du  
déploiement d'une plateforme de gestion de  
bibliothèque académique, en architecture  
microservices.

## RÉALISÉ PAR

DIALLO Salou  
COLY Ousmane  
OMAR Moussa  
DIA Adjii Ndioba  
DIA Amadou Daouda  
DIACK Mariam Bocar  
DIAKITE Mamadou Daye

## DÉTAILS

Filière : Master 1 Intelligence Artificielle  
Dakar Institute of Technology  
Date : Juillet 2026

## 01 Présentation du projet

Le Dakar Institute of Technology (DIT) gère actuellement sa bibliothèque académique de manière manuelle, ce qui pose plusieurs problèmes : difficulté de suivi des livres, absence de statistiques fiables, gestion inefficace des emprunts et manque d'accès numérique pour les étudiants.

Ce projet propose une plateforme web moderne, la « Bibliothèque Numérique DIT », permettant de digitaliser l'ensemble des opérations de la bibliothèque : gestion des livres, des utilisateurs et des emprunts. L'application repose sur une architecture microservices, conteneurisée avec Docker, et déployée automatiquement via un pipeline CI/CD Jenkins.

### Objectifs du projet

- Remplacer la gestion papier/manuelle par un système numérique centralisé.
- Permettre la recherche rapide d'un livre par titre, auteur ou ISBN.
- Distinguer les types d'utilisateurs (Étudiant, Professeur, Personnel administratif).
- Suivre les emprunts en cours et détecter automatiquement les retards.
- Automatiser le déploiement de l'application à chaque mise à jour du code.

### Technologies utilisées

| COMPOSANT              | TECHNOLOGIE             |
|------------------------|-------------------------|
| Backend                | FastAPI (Python 3.11)   |
| Frontend               | React + Vite            |
| Base de données        | PostgreSQL 15           |
| Conteneurisation       | Docker + Docker Compose |
| Serveur web (frontend) | Nginx                   |
| CI/CD                  | Jenkins                 |
| Gestion de version     | Git + GitHub            |

## 02 Architecture du système

L'application est construite selon une architecture microservices : chaque fonction métier (livres, utilisateurs, emprunts) est isolée dans un service indépendant, avec sa propre base de code, son propre Dockerfile, et communique avec les autres via des appels API REST en HTTP/JSON.

| COMPOSANT            | PORT | RÔLE                                                                      |
|----------------------|------|---------------------------------------------------------------------------|
| Frontend             | 3000 | Interface web React (Vite), servie par Nginx, point d'entrée utilisateur. |
| Service Livres       | 8000 | API FastAPI gérant le CRUD des livres et la recherche.                    |
| Service Utilisateurs | 8001 | API FastAPI gérant les comptes et profils utilisateurs.                   |
| Service Emprunts     | 8002 | API FastAPI gérant les emprunts, retours et retards.                      |
| PostgreSQL           | 5432 | Base de données relationnelle partagée, un schéma par service.            |

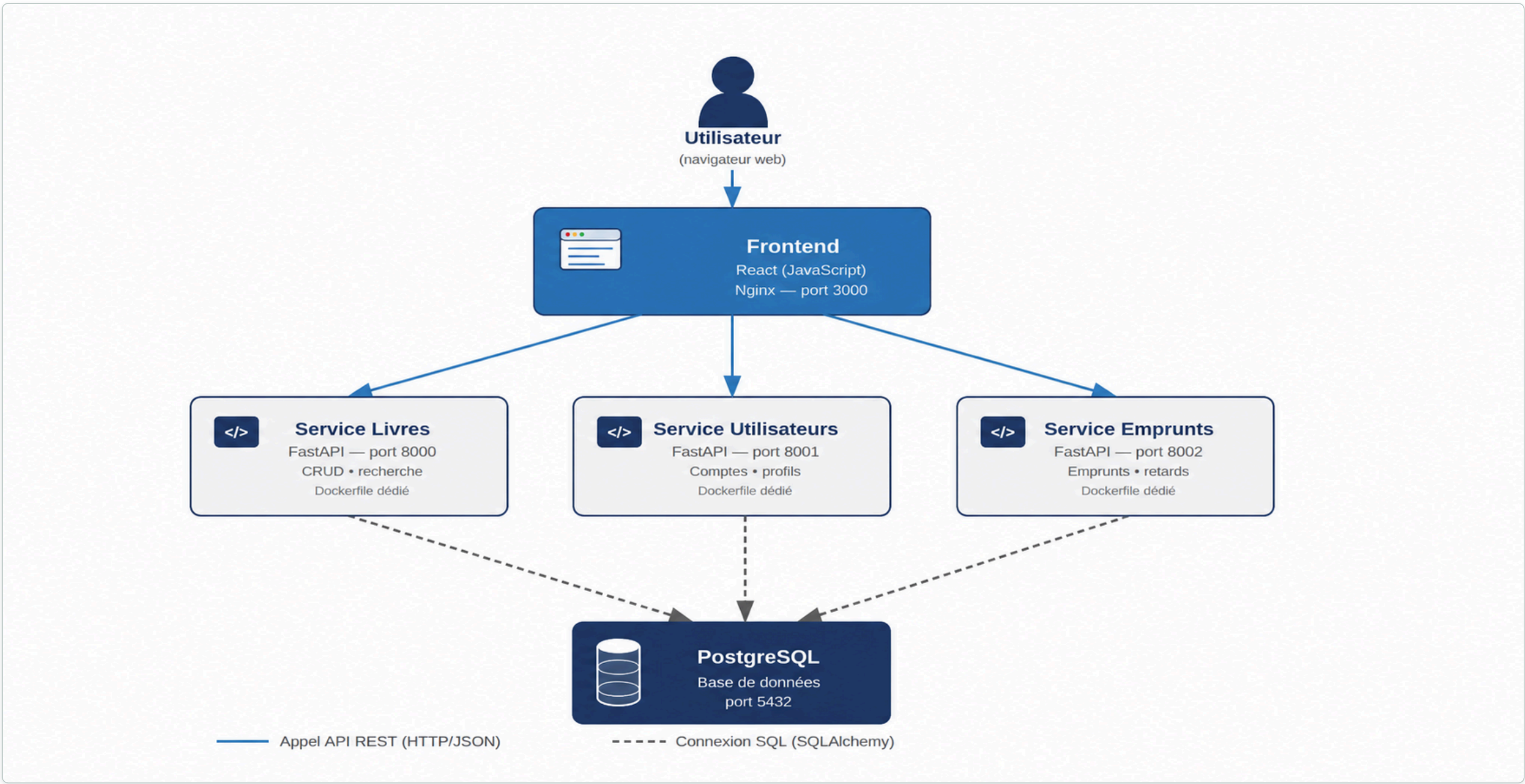


Figure 1. Schéma de l'architecture microservices : flux d'appels API (traits pleins) et connexions à la base de données (traits pointillés).

Chaque microservice backend possède sa propre image Docker (construite depuis un Dockerfile dédié) et communique avec la base PostgreSQL via une URL de connexion (SQLAlchemy). Le frontend React, compilé et servi par un conteneur Nginx, effectue des appels `fetch()` vers les 3 API backend. L'ensemble est orchestré par Docker Compose, qui définit le réseau interne, les volumes persistants et l'ordre de démarrage des services.

**Flux de communication simplifié**

- Le navigateur charge le frontend (port 3000).
- Le frontend appelle directement les API Livres (8000), Utilisateurs (8001) et Emprunts (8002).
- Chaque service interroge sa base de données PostgreSQL (port 5432) via SQLAlchemy.
- Les services sont indépendants : aucun n'appelle directement un autre microservice.



## 03 Description des microservices

### 3.1 Service Livres (port 8000)

Développé en FastAPI avec SQLAlchemy pour la persistance PostgreSQL. Il expose les endpoints suivants :

| MÉTHODE | ROUTE                | DESCRIPTION                                                           |
|---------|----------------------|-----------------------------------------------------------------------|
| GET     | /livres              | Liste tous les livres enregistrés.                                    |
| POST    | /livres              | Ajoute un nouveau livre (titre, auteur, ISBN).                        |
| PUT     | /livres/{id}         | Modifie les informations d'un livre existant.                         |
| DELETE  | /livres/{id}         | Supprime un livre par son identifiant.                                |
| GET     | /livres/recherche?q= | Recherche un livre par titre, auteur ou ISBN (insensible à la casse). |

#### Modèle de données

| CHAMP      | TYPE                                   |
|------------|----------------------------------------|
| id         | Integer (clé primaire)                 |
| titre      | String (obligatoire)                   |
| auteur     | String (obligatoire)                   |
| isbn       | String (unique, obligatoire)           |
| disponible | Integer (1 = disponible, 0 = emprunté) |

### 3.2 Service Utilisateurs (port 8001)

Gère les comptes des trois types d'utilisateurs : Étudiant, Professeur et Personnel administratif.

| MÉTHODE | ROUTE              | DESCRIPTION                                                          |
|---------|--------------------|----------------------------------------------------------------------|
| GET     | /utilisateurs      | Liste tous les utilisateurs.                                         |
| POST    | /utilisateurs      | Crée un nouvel utilisateur (avec vérification d'unicité de l'email). |
| GET     | /utilisateurs/{id} | Consulte le profil détaillé d'un utilisateur.                        |
| PUT     | /utilisateurs/{id} | Modifie les informations d'un utilisateur.                           |
| DELETE  | /utilisateurs/{id} | Supprime un utilisateur.                                             |

#### Modèle de données

| CHAMP            | TYPE                                                     |
|------------------|----------------------------------------------------------|
| id               | Integer (clé primaire)                                   |
| nom              | String (obligatoire)                                     |
| prenom           | String (obligatoire)                                     |
| email            | String (unique, obligatoire)                             |
| type_utilisateur | String (obligatoire) — Etudiant / Professeur / Personnel |

### 3.3 Service Emprunts (port 8002)

Gère le cycle de vie d'un emprunt, de la création au retour, ainsi que la détection automatique des retards.

| MÉTHODE | ROUTE                      | DESCRIPTION                                                               |
|---------|----------------------------|---------------------------------------------------------------------------|
| POST    | /emprunts                  | Enregistre un nouvel emprunt (livre, utilisateur, date de retour prévue). |
| PUT     | /emprunts/{id}/retour      | Marque un emprunt comme retourné (horodatage automatique).                |
| GET     | /emprunts                  | Retourne l'historique complet des emprunts.                               |
| GET     | /emprunts/utilisateur/{id} | Liste les emprunts d'un utilisateur donné.                                |
| GET     | /emprunts/retards          | Retourne les emprunts en retard (date dépassée et non retournés).         |

#### Modèle de données

| CHAMP                 | TYPE                                                |
|-----------------------|-----------------------------------------------------|
| id                    | Integer (clé primaire)                              |
| livre_id              | Integer (obligatoire — référence au livre emprunté) |
| utilisateur_id        | Integer (obligatoire — référence à l'emprunteur)    |
| date_emprunt          | DateTime (par défaut : date/heure actuelle)         |
| date_retour_prevue    | DateTime (obligatoire)                              |
| date_retour_effective | DateTime (optionnel — rempli au retour)             |

Les trois services partagent la même base PostgreSQL et créent automatiquement leurs tables au démarrage (SQLAlchemy `Base.metadata.create_all` ). Chaque service attend

activement que la base soit disponible avant de démarrer (fonction `wait_for_db()` ), afin d'éviter les erreurs de connexion lors du lancement simultané des conteneurs.



4.1 Un Dockerfile par service

Chaque microservice backend possède son propre `Dockerfile`. Voici le modèle utilisé pour les trois services FastAPI (livres, utilisateurs, emprunts) :

| INSTRUCTION             | RÔLE                                            |
|-------------------------|-------------------------------------------------|
| FROM python:3.11-slim   | Image de base légère Python 3.11                |
| WORKDIR /app            | Répertoire de travail dans le conteneur         |
| COPY requirements.txt . | Copie des dépendances                           |
| RUN pip install ...     | Installation des dépendances                    |
| COPY . .                | Copie du code source                            |
| EXPOSE 800X             | Port exposé par le service (8000 / 8001 / 8002) |
| CMD [uvicorn ...]       | Démarrage du serveur FastAPI                    |

4.2 Frontend React — build multi-étapes

Le Dockerfile du frontend utilise un **build multi-étapes**, pour ne pas embarquer Node.js ni les outils de build dans l'image finale :

| ÉTAPE                    | DESCRIPTION                                                                                                                |
|--------------------------|----------------------------------------------------------------------------------------------------------------------------|
| Builder — node:20-alpine | Installation des dépendances ( <code>npm install</code> ) et build de production React/Vite ( <code>npm run build</code> ) |
| Serveur — nginx:alpine   | Récupère uniquement le résultat du build ( <code>dist/</code> ) et sert les fichiers statiques compilés                    |

Cette approche produit une image finale beaucoup plus légère (quelques Mo, uniquement Nginx + fichiers statiques) qu'une image contenant Node.js et `node_modules`.

4.3 Orchestration avec Docker Compose

Le fichier `docker-compose.yml` orchestre les 5 conteneurs et définit leurs interdépendances :

- Le service `db` (PostgreSQL) démarre en premier.
- Les 3 services FastAPI attendent activement que PostgreSQL soit prêt avant de démarrer (fonction `wait_for_db()` ), pour éviter les erreurs de connexion au lancement simultané.
- Un volume nommé `postgres_data` persiste les données de la base entre les redémarrages.
- Tous les services communiquent sur le même réseau Docker interne créé automatiquement par Compose, et exposent leurs ports vers l'hôte (3000, 8000, 8001, 8002, 5432).

4.4 Conteneurs en production

Vue Docker Desktop après `docker compose up -d --build` : les 5 conteneurs sont actifs, avec leurs ports mappés et leurs logs de démarrage (chaque service FastAPI confirme `Uvicorn running on ...` une fois prêt) :

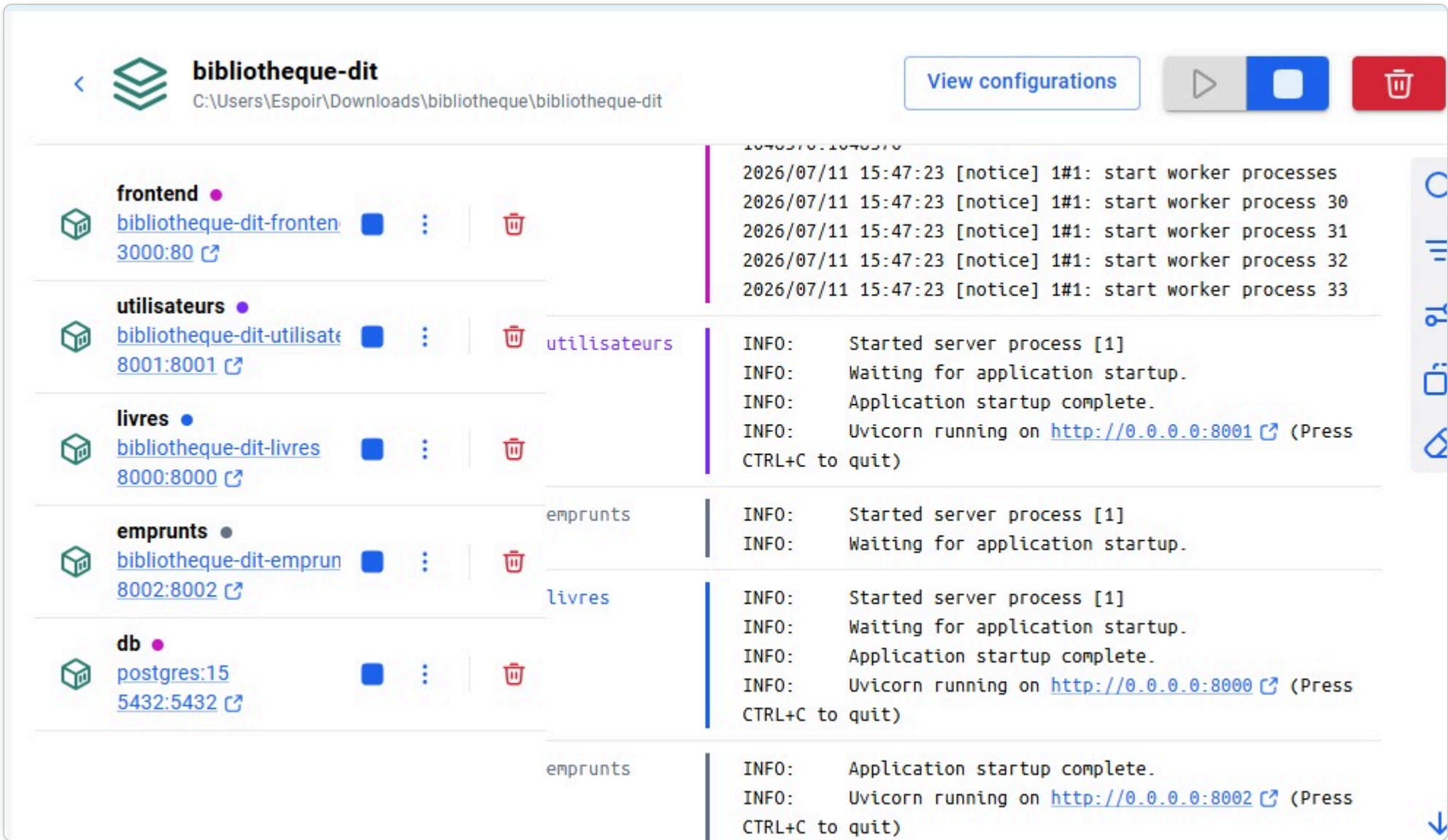


Figure 2. Les 5 conteneurs du projet actifs dans Docker Desktop : frontend (3000:80), utilisateurs (8001:8001), livres (8000:8000), emprunts (8002:8002) et db (5432:5432).

05 Explication du pipeline CI/CD

Le déploiement est automatisé via un pipeline Jenkins défini dans un `Jenkinsfile` à la racine du projet. Il se compose de trois étapes principales (stages) exécutées séquentiellement :

| # | ÉTAPE                   | ACTION RÉALISÉE                                                                                                                                                         |
|---|-------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1 | Récupération du code    | <code>checkout scm</code> — récupère la dernière version du code source depuis le dépôt GitHub configuré dans le job Jenkins.                                           |
| 2 | Build des images Docker | <code>docker compose build</code> — reconstruit les images Docker de chaque service (livres, utilisateurs, emprunts, frontend) à partir de leurs Dockerfile respectifs. |
| 3 | Déploiement             | <code>docker compose up -d</code> — démarre (ou redémarre) l'ensemble des conteneurs en arrière-plan avec la configuration définie dans docker-compose.yml.             |

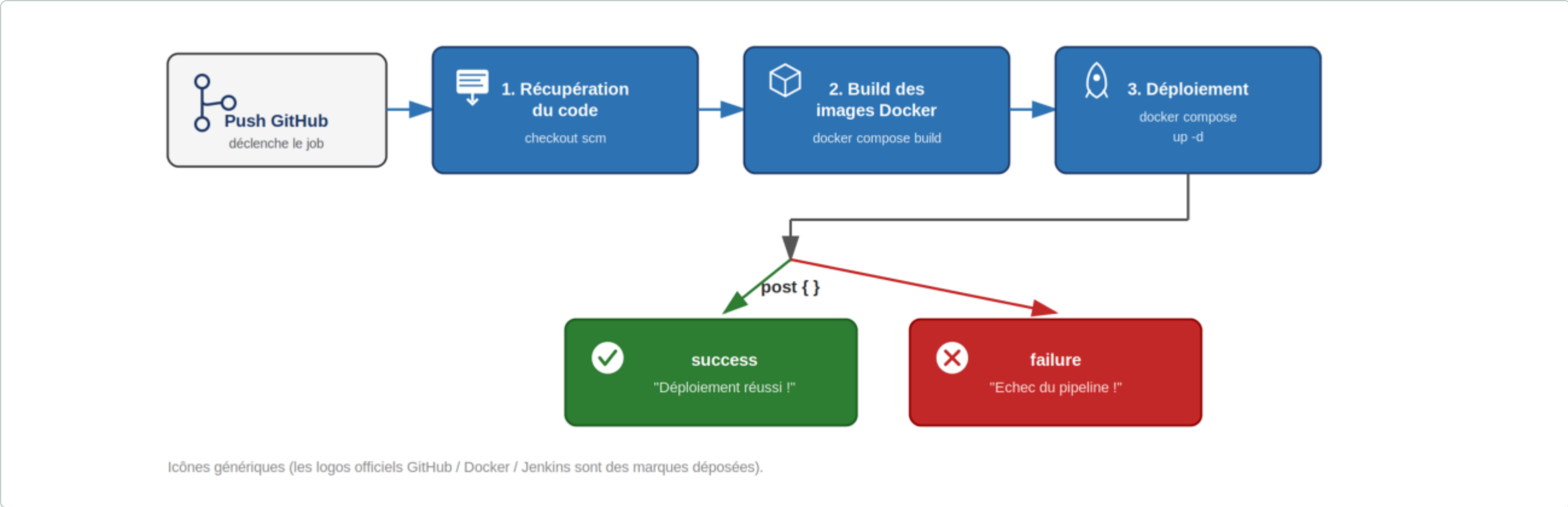


Figure 3. Flux du pipeline Jenkins : les trois étapes s'exécutent séquentiellement, suivies du bloc `post {}` qui affiche un message de succès ou d'échec selon le résultat.

Un bloc `post` gère le résultat final du pipeline : un message de succès s'affiche si toutes les étapes se sont bien déroulées, ou un message d'échec en cas de problème à n'importe quelle étape.

Avantages de cette automatisation

- Élimination des étapes manuelles répétitives (clone, build, déploiement).
- Déploiement reproductible et identique à chaque exécution.
- Historique des builds consultable dans l'interface Jenkins (succès/échec, logs détaillés).
- Base extensible pour ajouter plus tard des tests automatisés ou une analyse de qualité de code (SonarQube).

Tableau de bord, offrant une vue d'ensemble en temps réel du catalogue, des utilisateurs, des emprunts en cours et des retards à traiter :

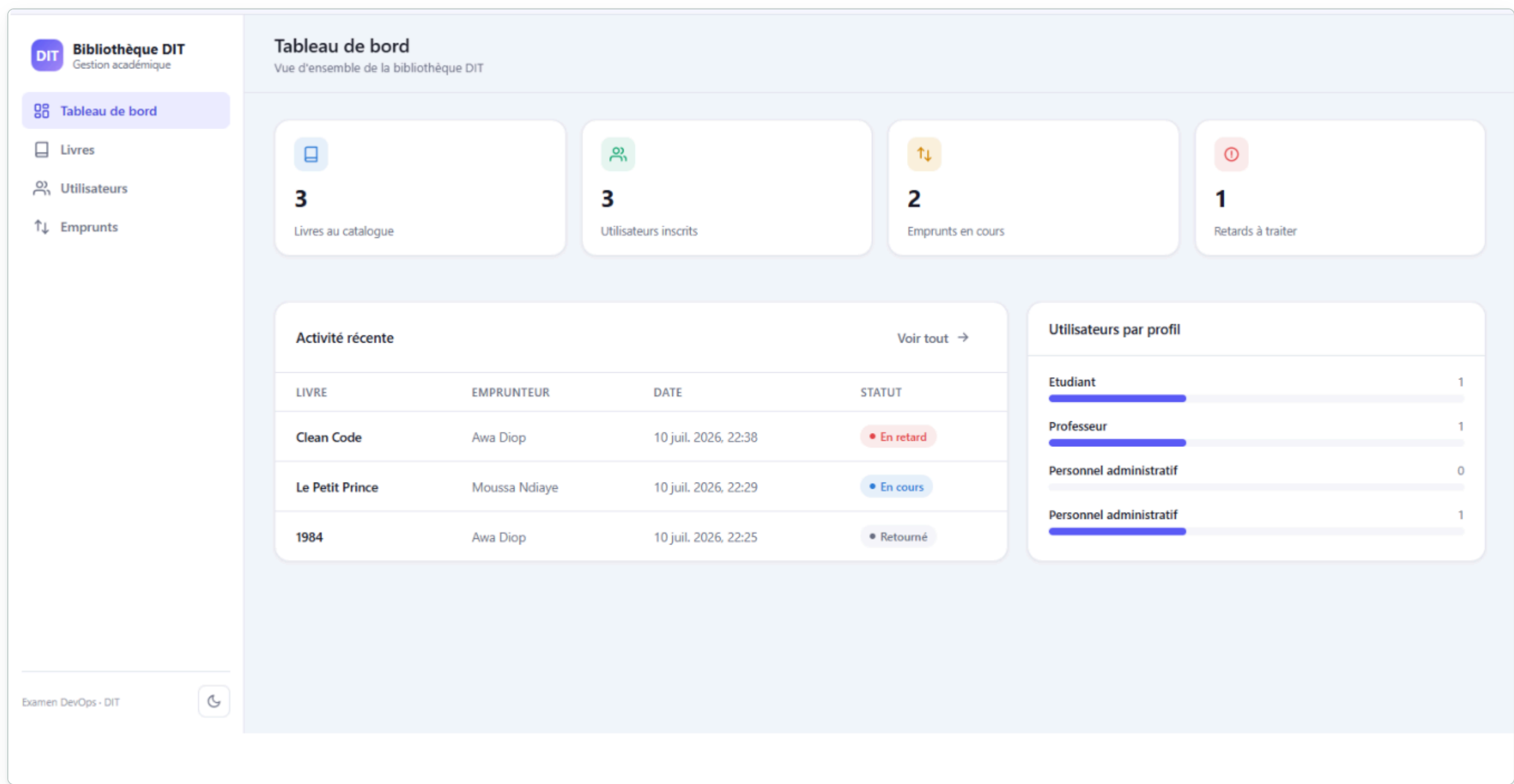


Figure 4. Tableau de bord : statistiques globales et répartition des utilisateurs par profil.



Gestion des livres : liste du catalogue avec recherche, ajout, modification et suppression :

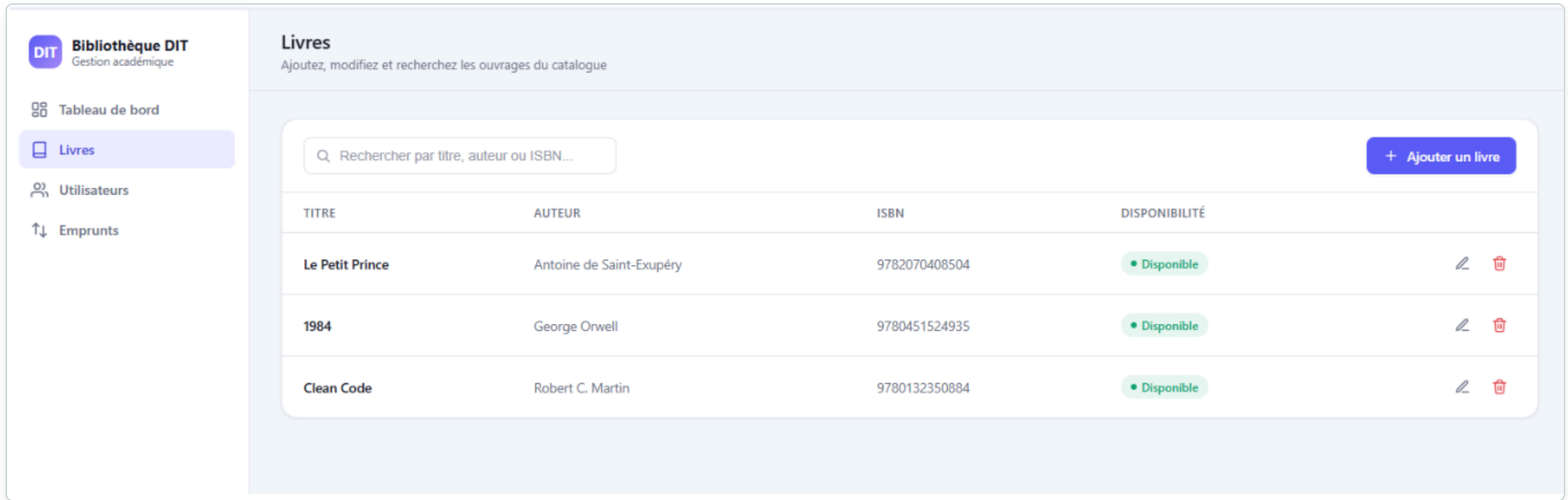


Figure 5. Page « Livres » : catalogue avec statut de disponibilité et actions Éditer / Supprimer.

Gestion des utilisateurs, avec distinction des trois profils (Étudiant, Professeur, Personnel administratif) :

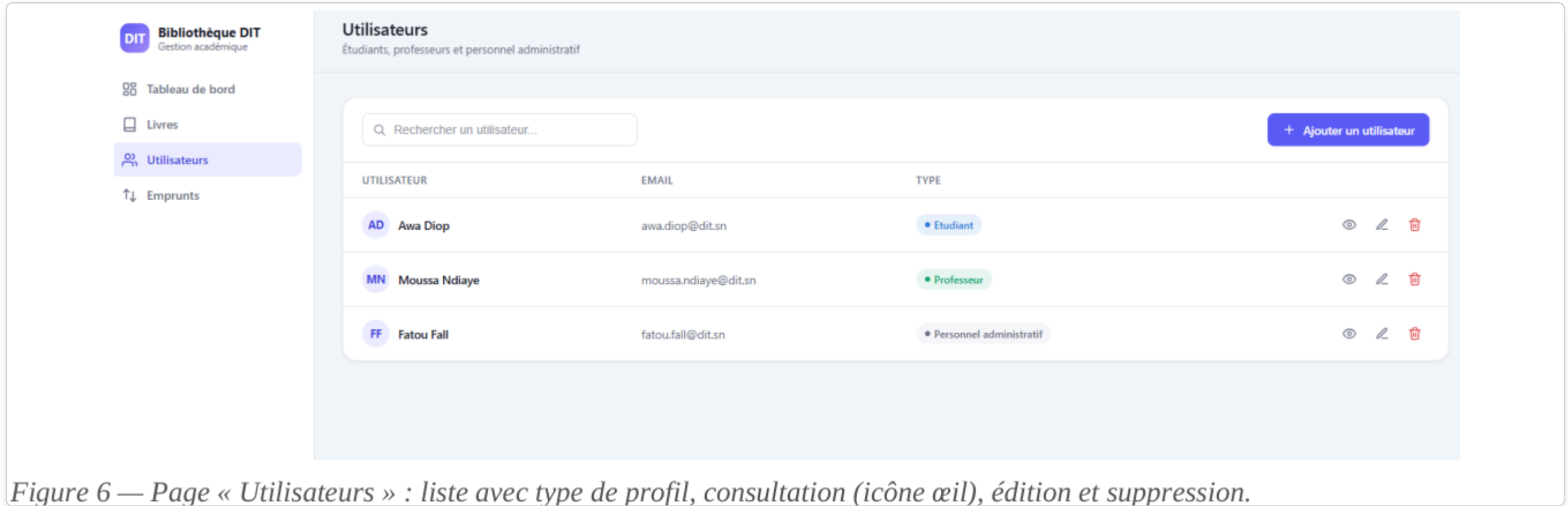


Figure 6 — Page « Utilisateurs » : liste avec type de profil, consultation (icône œil), édition et suppression.

Figure 6. Page « Utilisateurs » : liste avec type de profil, consultation (icône œil), édition et suppression.

Suivi des emprunts : historique complet avec détection automatique des retards (statut « En retard » affiché en rouge) :

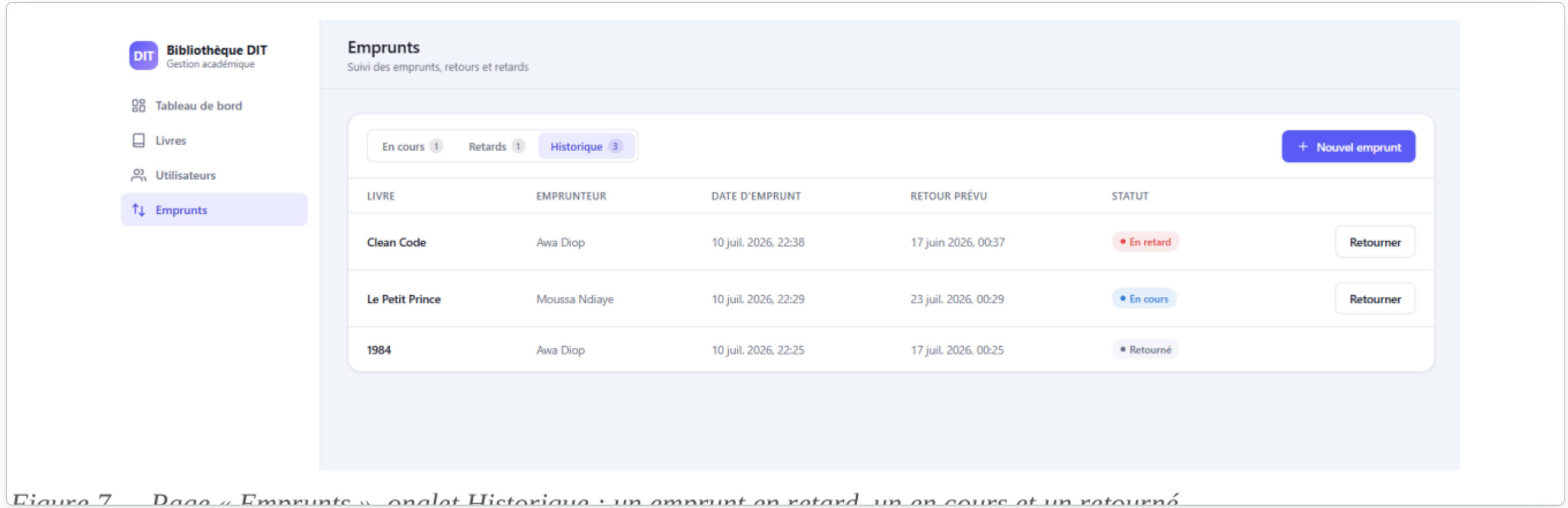


Figure 7. Page « Emprunts », onglet Historique : un emprunt en retard, un en cours et un retourné.

Exécution du pipeline Jenkins : les 5 étapes (Consultez SCM, Récupération du code, Créer des images Docker, Déploiement, Actions sur la publication) se sont déroulées avec succès :

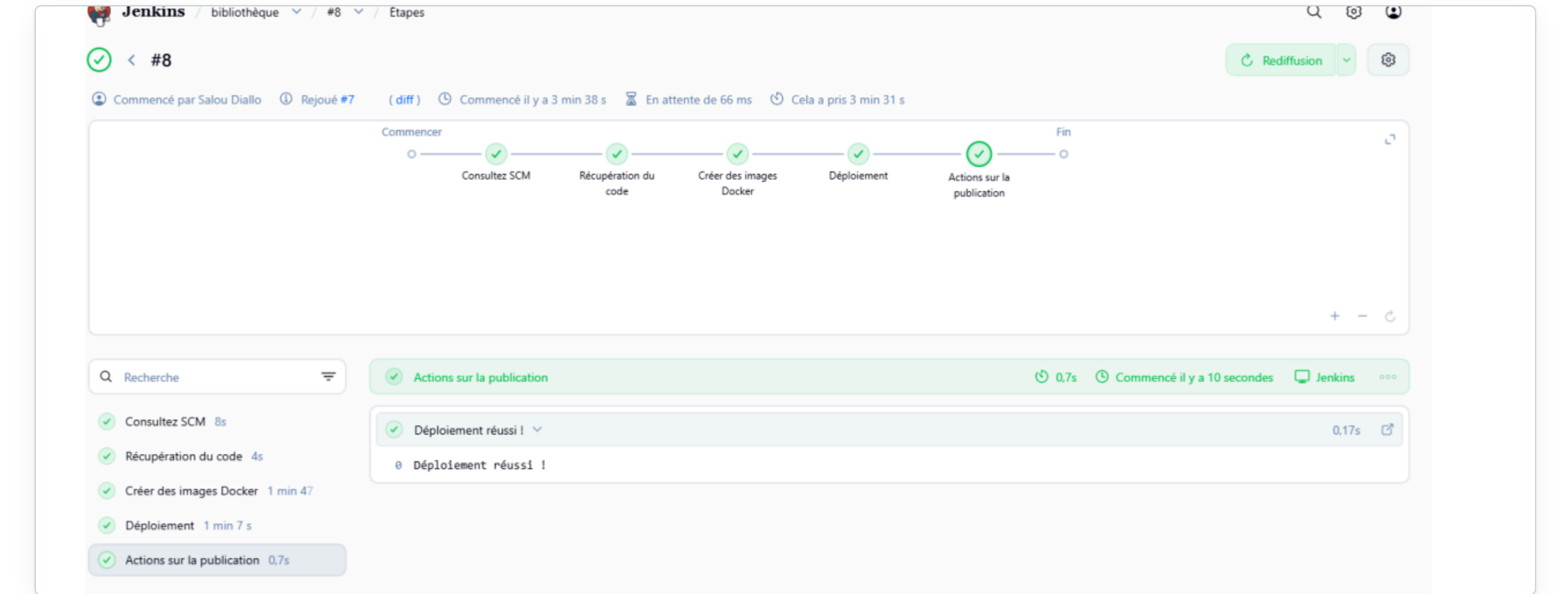


Figure 8. Build Jenkins #8 : pipeline exécuté avec succès, message « Déploiement réussi ! » confirmé dans les logs.

Ce projet nous a permis de mettre en pratique les concepts fondamentaux du DevOps dans un contexte réel et concret.

Compétences acquises

- Conception d'une architecture microservices avec FastAPI
- Développement d'API REST complètes (CRUD)
- Conteneurisation avec Docker et Docker Compose
- Mise en place d'un pipeline CI/CD avec Jenkins
- Gestion de projet en équipe avec Git et GitHub (branches `feature/*` , pull requests)

Difficultés rencontrées

- **Synchronisation de PostgreSQL au démarrage** — résolu avec `wait_for_db()` .
- **Intégration React ↔ FastAPI** — appels bloqués par le navigateur, résolu avec `CORSMiddleware` sur les trois services.
- **Dockerfile pour React/Vite** — résolu avec un build multi-étapes (Node puis Nginx).
- **Docker sous Windows** — WSL2 non activé par défaut, résolu en activant « Plateforme d'ordinateur virtuel » et l'hyperviseur.
- **Coordination des branches Git** — travail réuni dans une branche d'intégration avant fusion sur `main` via pull requests.

Les autres membres de l'équipe peuvent compléter cette liste avec les difficultés spécifiques qu'ils ont rencontrées sur leur propre service.

Résultat final

L'application **Bibliothèque Numérique DIT** est pleinement fonctionnelle, avec :

- 3 microservices backend indépendants (Livres, Utilisateurs, Emprunts)
- 1 interface frontend moderne en React, couvrant l'intégralité des fonctionnalités demandées
- 1 base de données PostgreSQL persistante
- Un déploiement entièrement automatisé via Docker Compose
- Un pipeline CI/CD opérationnel avec Jenkins